Práctico 13 - Búsqueda y Ordenación

Programación Imperativa
InCo - Facultad de Ingeniería, Udelar

1. Sean las siguientes declaraciones:

```
const
    MAX = . . .;  {entero mayor estricto que 0}
type
      ArregloEnteros = array [1..MAX] of integer;
```

   (a) Escriba la función `buscarElemento` que determina si el elemento `e` se encuentra en el arreglo `a`.

   ```
   function buscarElemento(e:integer; a:arregloEnteros):boolean
   ```

   (b) Escriba la función `unSoloMayor` que determina si el arreglo `a` contiene un único elemento mayor que el valor `v`.

   ```
   function unSoloMayor(v:integer; a:arregloEnteros):boolean
   ```

   (c) Escriba la función `todosMayores` que determina si todos los elementos del arreglo `a` son mayores que el valor `v`.

   ```
   function todosMayores(v:integer; a:arregloEnteros):boolean
   ```

   (d) Escriba la función `cuantosMayores` que determina cuántos elementos del arreglo `a` son mayores que el valor `v`.

   ```
   function cuantosMayores(v:integer; a:arregloEnteros):integer
   ```

   (e) Escriba la función `ultimaOcurrencia` que determina la última posición en el arreglo `a` en la cual ocurre el elemento `e`. Si `e` no se encuentra en `a`, la función devuelve 0.

   ```
   function ultimaOcurrencia(e:integer; a:arregloEnteros):integer
   ```

2. Sean las siguientes declaraciones:

```
const
    MAXCOL = . . .; {entero mayor estricto que 0}
    MAXFIL = . . .; {entero mayor estricto que 0}
type
    MatrizEnteros = array [1..MAXFIL, 1..MAXCOL] of integer;
```

   Escriba la función `buscarElementoMatriz` que determina si el elemento `e` ocurre en la matriz `m`. Puede invocar las funciones del ejercicio 1. Para ello puede suponer que las constantes `MAXCOL` y `MAX` coinciden.

```
function buscarElementoMatriz(e:Integer; m:matrizEnteros):boolean
```

3. Sean las siguientes declaraciones:

```
const
    MAX = . . .; {entero mayor estricto que 0}
type
      TComparacion = (mayor, menor, igual);
      TTexto = record
                     nombre : array [1 .. MAX] of char;
                     tope     : 0 .. MAX;
                 end;
      ArregloNombres = array [1..MAX] of TTexto;
```

(a) Escriba la función `comparacionNombres` que dados dos nombres `n1` y `n2` devuelva la comparación entre ellos. La comparación es en base al orden lexicográfico.

```
function comparacionNombres(n1, n2:TTexto):TComparacion
```

(b) Escriba la función `buscarNombre` que devuelve `TRUE` si el nombre `n` ocurre dentro del arreglo `a` y `FALSE` en caso contrario. El arreglo de nombres `a` está ordenado en forma lexicográfica y creciente. ¿Realizará búsqueda lineal o binaria? ¿Por qué?

```
function buscarNombre(n:TTexto; a:arregloNombres):boolean
```

(c) Para la parte anterior, ¿consideró si había nombres repetidos en el arreglo? Si no lo consideró, ¿qué cambia en su código si los hay? ¿Qué cambiaría si hay nombres repetidos y se le pide encontrar la primera ocurrencia de un nombre?

4. Sean las siguientes declaraciones:

```
const
    MAXE  = . . .; {entero mayor estricto que 0}
    MAXC = . . .;  {entero mayor estricto que 0}
    MAX = . . .; {entero mayor estricto que 0}

type
      TTexto = record
                    caracteres : array [1 .. MAX] of char;
                    tope     : 0 .. MAX;
                end;

     TEstudiante = record
                    nombre     : TTexto;
                    generacion : integer;
                end;

     TEstudiantes = record
                             estudiante : array [1 .. MAXE] of TEstudiante;
                             tope : 0 .. MAXE;
                         end;

      TCarrera = record
                       nombre : TTexto;
                       estudiantes : TEstudiantes;
                    end;

     TBedelias = array [1 .. MAXC] of TCarrera;
```

que representan los estudiantes que están inscriptos a una carrera de grado en facultad. Un elemento de tipo `TBedelias` tiene ordenadas las carreras en forma lexicográfica y creciente por su nombre. Luego, por cada carrera, la lista de estudiantes se encuentra ordenada por el año de ingreso en forma creciente.

(a) Escriba la función `buscarEstudiante` que determina si el estudiante de nombre `nombreE` está inscripto a la carrera de nombre `nombreC`. Para comparar los nombres de las carreras y de los estudiantes, puede usar la función definida en el ejercicio 3.a.

```
function buscarEstudiante(nombreE:TTexto; nombreC:TTexto; bed:TBedelias):boolean
```

(b) En la parte anterior, ¿qué tipos de búsquedas tuvo que realizar? Explique.

(c) Si ahora asumimos que dentro de la misma carrera y el mismo año de ingreso, los nombres de los estudiantes están ordenados en forma lexicográfica y creciente.

Escriba el procedimiento `buscarEstudiantesGeneracion` que devuelva en `bGen` todos los estudiantes que ingresaron en el año `gen`. ¿Qué tipo de búsqueda debe realizar?.

El orden de `bed` es el definido en el ejercicio. El parámetro `bGen` debe mantener dicho orden. Si no hay estudiantes que ingresaron en tal año para una carrera, la lista de estudiantes para dicha carrera debe quedar vacía.

```
procedure buscarEstudiantesGeneracion(gen:integer; bed:TBedelias; var bGen:TBedelias)
```

5. Sean las siguientes declaraciones:

```
const
    MAX = . . .; {entero mayor estricto que 0}
type
      Rango = 1 .. MAX;
      ArregloEnteros = array [rango] of integer;
```

   (a) Escriba la función `estaOrdenado` que determina si el arreglo `a` está ordenado en forma ascendente.

   ```
   function estaOrdenado(a:arregloEnteros):boolean
   ```

   (b) Escriba el procedimiento `ordenarPosicion` que recibe un arreglo `a` ordenado parcialmente entre las posiciones `1` y `pos-1` de forma ascendente y re-inserta el elemento que originalmente se encuentra en la posición `pos` de forma ordenada entre las posiciones `1` y `pos`. Notar que al finalizar el procedimiento el arreglo debe quedar parcialmente ordenado entre las posiciones `1` y `pos`.

   ```
   procedure ordenarPosicion(pos:rango; var a:arregloEnteros)
   ```

   (c) Escriba el procedimiento `ordenarArregloEnteros` que ordena de forma ascendente el arreglo `a` que recibe por parámetro. Utilice la parte b.

   ```
   procedure ordenarArregloEnteros(var a:arregloEnteros)
   ```

6. Sean las siguientes declaraciones:

```
const
    MAXF = . . .; {entero mayor estricto que 0}
    MAXC = . . .; {entero mayor estricto que 0}

type
      RangoF = 1 .. MAXF;
      RangoC = 1 .. MAXC;

      TVariante = (rojo, azul, verde);

      TFicha = record
                    tipo       :  TVariante;
                    puntaje  : integer;
                  end;
      TTablero = array [rangoF, rangoC] of TFicha;
```

   Escriba el procedimiento `ordenarTablero` que ordena por el método de inserción las fichas del tablero de forma ascendente según los siguientes criterios:

   - Según color: rojo $<$ azul $<$ verde
   - Si hay empate de color se ordena en forma ascendente por el puntaje

   El recorrido de la matriz se define primero por el índice de la fila de menor a mayor y luego por el índice de la columna de menor a mayor.

```
procedure ordenarTablero(var t:TTablero);
```

7. Escriba un programa donde el usuario ingrese una secuencia de enteros positivos y la salida sea la secuencia ordenada en forma descendente utilizando el método del ordenamiento por inserción. La secuencia ingresada por el usuario es de largo variable y termina con el número `-1`. Si la secuencia es vacía, el programa no debe imprimir nada. Puede asumir que como máximo el usuario ingresará una cantidad `N` de números, siendo `N` una constante declarada al inicio del programa.

8. Dadas las siguientes declaraciones:

```
const
    MAX = . . .;  {entero mayor estricto que 0}
type
      Rango = 1 .. MAX;
      ArregloCaracteres = array [rango] of char;
```

(a) Escriba la función `maximoArregloCar` que devuelve el indice del máximo carácter según la ordenación de la tabla ASCII ubicado entre las posiciones `ini` y `fin`. Puede asumir que `ini` es menor o igual a `fin`.

```
function maximoArregloCar(ini, fin:rango; a:arregloCaracteres):rango
```

(b) Escriba el procedimiento `ordenarArregloCaracteres` que ordena el arreglo `a` pasado por parámetro de forma ascendente utilizando el método de ordenación por selección.

```
procedure ordenarArregloCaracteres(var a : arregloCaracteres);
```

9. Sean las siguientes declaraciones:

```
const
    MAX = . . .; {entero mayor estricto que 0}
type
      RangoCalificacion   = 1..12;
      TAsignatura = (Criptografia,Programacion,Calculo);
      TEstudiante = record
                              idEstudiante : integer;
                              asignatura : TAsignatura;
                              calificacion : RangoCalificacion;
                          end;
      Rango = 1 .. MAX;
      EstudiantesAsignatura = array [rango] of TEstudiante;
```

que representa las calificaciones obtenidas por los estudiantes en las asignaturas. El estudiante se identifica por el número `idEstudiante`. Puede suponer que un estudiante puede cursar varias asignaturas y una única vez.

Escriba el procedimiento `ordenarPorAsignatura` que recibe un arreglo de estudiantes `e` con la calificación obtenida en las asignaturas y lo ordena en forma ascendente según los siguiente criterios, utilizando el método de ordenación por selección:

- En primer lugar por el orden de la asignatura, definido en el tipo.
- En segundo lugar por la calificación.
- En tercer lugar por la identificación.

Se sugiere realizar un procedimiento auxiliar que compare dos estudiantes y devuelva si uno es posterior al otro.

```
procedure ordenarPorAsignatura(var e:EstudiantesAsignatura)
```

10. Sean las siguientes declaraciones:

```
const
    MAX = . . .; {entero mayor estricto que 0}
type
      Rango = 1 .. MAX;
      ArregloNumeros = array [rango] of integer;
```

(a) Escriba el procedimiento `ordenarParesImpares` que recibe un arreglo de enteros `a` desordenado y ordena de forma ascendente los elementos que ocupan posiciones pares en el arreglo y de forma descendente los elementos que ocupan las posiciones impares del arreglo.

```
procedure ordenarParesImpares(var a:arregloNumeros)
```

(b) Modifique su implementación del procedimiento anterior para que el ordenamiento de los elementos en las posiciones pares e impares se realice de forma simultánea.

11. (a) Escriba un programa donde el usuario ingresa números enteros positivos, terminando la secuencia con el número `-1`. Puede asumir que como máximo el usuario ingresará una cantidad `N` de números, siendo `N` una constante declarada al inicio del programa. Luego, el usuario ingresa otra secuencia de números enteros positivos que termina en `-1`. El programa debe mostrar, por cada número de la segunda secuencia, si éste pertenece a la primera secuencia.

Ejemplo:

```
1 9 100 20 88 100 1 -1
6
El número 6 no pertenece a la secuencia.
9
El número 9 pertenece a la secuencia
100 2
El número 100 pertenece a la secuencia
El número 2 no pertence a la secuencia
-1
```

Analice las distintas formas de implementar dicho programa teniendo en cuenta los algoritmos de búsquedas y ordenación vistos en el curso y justifique la elección de los algoritmos empleados, de forma de tener en cuenta el tiempo de ejecución del programa.

(b) Modifique su programa para que imprima la cantidad de comparaciones que fueron necesarias para determinar si un número pertenece a la secuencia. ¿Cómo relaciona la cantidad de comparaciones con los algoritmos de búsqueda vistos en el curso y la forma de implementar su programa?